Cor Unum V1 Specification:

Context Doc: cor_unum subfunctionality

1. What cor_unum is in this system
   cor_unum is a live music ingestion and curation subsystem inside Noctem, focused on Vancouver events.
   It has its own DB namespace (cu_* tables), ingestion pipeline, admin-style web UI, and review workflows, while remaining integrated into the main Flask app and shared SQLite database.

Primary surface entrypoints:

- Web: /cor-unum and related pages in noctem/web/templates/cor_unum*.html
- Backend routes: noctem/web/app.py
- Core ingestion logic: noctem/ingestion/*

1. Current architecture (as implemented)

Data layer
Defined in noctem/ingestion/schema.py, initialized via noctem/db.py (init_db() calls _init_cu_tables()).

Key tables:

- cu_events, cu_venues, cu_artists
- cu_event_performers (event ↔ artist)
- cu_event_sources (provenance + fingerprint per source)
- cu_source_registry (control plane for scanners)
- cu_ingestion_runs (run history + raw summary JSON)
- cu_artist_tags (city tags, currently YVR)
- cu_duplicate_ignores (suppression for reviewed duplicate candidates)
- cu_artist_link_candidates (manual review queue for discovered social URLs)
- cu_favorites exists as a stub and is not wired into routes/services yet

Ingestion pipeline
Implemented in noctem/ingestion/engine.py, scanner base in scanners.py, concrete scanners in scanner_impl.py.

Pipeline order:

1. event scanners (Ticketmaster, RA, AdmitOne, Eventbrite)
2. fingerprint scanners (SoundCloud, Instagram, Spotify)
3. internal janitors (artist/event duplicate candidate generation)

Event ingestion behavior
_process_one_event() in noctem/ingestion/engine.py:

- Computes deterministic fingerprint from normalized title + date (dedup.py)
- Applies source-scoped dedupe (exact + fuzzy on same source/date)
- Creates/updates event, venue, performers, source evidence
- Uses fallback venue "Out in the Wild" when venue is missing

Artist enrichment behavior
Fingerprint modules:

- noctem/ingestion/soundcloud.py
- noctem/ingestion/instagram.py
- noctem/ingestion/spotify.py
- Unified through noctem/ingestion/artist_fingerprints.py

Current signals are heuristic (keyword checks in fetched profile page text, e.g., vancouver/canada/yvr).
Instagram/Spotify also support auto-discovery via noctem/ingestion/link_discovery.py.

Service + API layer
noctem/ingestion/service.py provides route-safe functions for:

- source control (enable/disable, refresh, clear errors)
- run history
- paginated DB views
- detail views (event, artist, venue)
- merge/alias operations
- duplicate rescan/ignore/merge

Routes are exposed in noctem/web/app.py under /api/cor-unum/*.

UI layer
Templates:

- Dashboard/control: cor_unum.html
- DB views: cor_unum_db.html
- Upcoming/events: cor_unum_upcoming.html, cor_unum_event.html
- Entity pages: cor_unum_artist.html, cor_unum_venue.html
- Duplicate review: cor_unum_duplicates.html
- Manual event entry: cor_unum_add_event.html
- Settings: cor_unum_settings.html

Test coverage
Primary test module: noctem/tests/test_cor_unum.py
Covers:

- schema creation/seeding/idempotency
- dedupe logic and source-scoped behavior
- service/API basics
- run summary recording
- Instagram/Spotify recheck/discovery behavior (with monkeypatched network calls)

End-to-end flow (current state)

1. Admin triggers source/class/full pipeline from the Cor Unum dashboard.
2. Scrapers/fingerprint/internal scanners run and write run summaries.
3. Event data lands in cu_* tables with provenance.
4. UI pages read paginated/detail endpoints.
5. Duplicates are reviewed in a dedicated page (merge or ignore).
6. Manual event creation and manual artist URL/locality edits are available.

Important constraints and gaps (current implementation)

1. Auth is effectively bypassed for Cor Unum admin actions.
    - In noctem/web/app.py, session bootstrap sets a default admin user.
2. Cross-source event dedupe is not automatic.
    - Dedupe is source-scoped; cross-source reconciliation relies on janitor candidates + manual merge.
3. Discovery candidate queue is partially latent.
    - cu_artist_link_candidates and review functions exist in link_discovery.py, but no corresponding web/API moderation flow is currently wired.
4. External dependency fragility exists.
    - SoundCloud relies on scraping a public client_id.
    - Event sources depend on third-party HTML/GraphQL contracts.
    - Playwright runtime/browser availability is required.
5. No scheduler orchestration for Cor Unum pipeline is present.
    - Runs are operator-triggered via UI/API.
6. Schema includes forward-looking stubs.
    - cu_favorites is present but unused in surfaced functionality.

Outline for next update report

Objective

- What outcome this Cor Unum update must deliver

Current-State Snapshot

- Short summary of pipeline, UI surfaces, and key constraints

Problems to Solve

- Ranked list (quality, reliability, UX, security)

Proposed Changes by Layer

- Data/schema
- Ingestion engine/scanners
- Service/API
- UI/workflows
- Testing/ops

Migration + Compatibility

- DB migration impacts, backfills, rollout safety

Success Metrics

- Ingestion success rate, duplicate precision, review throughput, mean time to fix source failures

Risks + Mitigations

- External source breakage, false merges, auth regressions, runtime dependencies

Execution Plan

- Phases, dependencies, and validation gates